Python Lessons

From the Basics to Object-Oriented Programming

16 lessons with examples and exercises

Table of Contents

Lesson 1: The print function and variables

Lesson 2: Input and type conversion

Lesson 3: Arithmetic operations

Lesson 4: Conditional statements (if)

Lesson 5: The for loop

Lesson 6: The while loop, break and continue

Lesson 7: Working with strings

Lesson 8: Lists

Lesson 9: Tuples and sets

Lesson 10: Dictionaries

Lesson 11: Functions

Lesson 12: Modules and libraries (import)

Lesson 13: Error handling (try / except)

Lesson 14: Working with files

Lesson 15: Object-oriented programming (1): Classes and objects

Lesson 16: Object-oriented programming (2): Inheritance

What comes after the basics?

Lesson 1: The print function and variables

The `print()` function is used to display text, numbers, or the results of calculations on the screen. We put the text between quotation marks inside the parentheses.

```
print("Welcome to the world of programming")
```

Variables

A variable is a space in the computer's memory. Think of it as a box in which we store data and give it a name so we can call it later.

- **Text (str):** written between quotation marks, e.g. "Khaled"
- **Integer (int):** written directly, e.g. 30
- **Decimal number (float):** e.g. 99.5
- **Boolean (bool):** either True or False

```
user_name = "Khaled"
user_age = 30
price = 99.5
is_student = True
print(type(user_age))  # <class 'int'>
```

Combining text with variables

We separate the fixed text from the variable with a comma `,` inside `print`.

```
my_name = "Mohammed"
my_country = "Algeria"
print("My name is", my_name)
print("I live in", my_country)
```

Output on the screen:

```
My name is Mohammed
I live in Algeria
```

Variable naming rules

- A name starts with an English letter or an underscore _, not with a number.
- It must not contain spaces (use `user_name` instead of `user name`).
- Python is case-sensitive: `Age` is different from `age`.

Exercise

- Store your name, your city, and your age in three variables, then print them on three separate lines.

Lesson 2: Input and type conversion

When Python reaches `input()`, it pauses the program and waits for the user to type something and press Enter.

```
user_name = input("Please enter your name: ")
print("Welcome,", user_name)
```

The problem with numbers and input()

Python treats anything the user types as text (str), even if it is a number. So if we try to add a number to text, a `TypeError` occurs. The solution is conversion using `int()` for integers or `float()` for decimals.

```
# Wrong way:
age = input("How old are you? ")
# age + 1 causes a TypeError

# Correct way: convert to a number
age = int(input("How old are you? "))
print("Next year you will be:", age + 1)
```

A simple calculator program

```
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))
total = num1 + num2
print("The sum of the two numbers is:", total)
```

Tip: The most commonly used conversion functions: `int()`, `float()`, and `str()`.

Exercise

- Ask the user for their height in centimeters (as a decimal number), then print it in meters.

Lesson 3: Arithmetic operations

Python supports all arithmetic operations, including useful ones such as integer division and remainder.

Operation	Symbol	Example	Result
Addition	+	17 + 5	22
Subtraction	-	17 - 5	12
Multiplication	*	17 * 5	85
Division	/	17 / 5	3.4
Integer division	//	17 // 5	3
Remainder (modulo)	%	17 % 5	2
Exponent (power)	**	17 ** 2	289

```
a = 17
b = 5
print(a + b)   # 22
print(a / b)   # 3.4
print(a // b)  # 3
print(a % b)   # 2
print(a ** 2)  # 289
```

Tip: The `%` operation is very useful: if `n % 2 == 0`, then the number is even.

Exercise

- Ask the user for a number, then print its square and its cube.

Lesson 4: Conditional statements (if)

In life we make decisions based on conditions, such as "if it is raining, I will take the umbrella." In Python we use `if` for the same purpose.

Comparison operators

Operator	Meaning
==	Exactly equal (two equals signs, not one)
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

if and else

```
score = 40
if score >= 50:
    print("Congratulations, you passed!")
else:
    print("Unfortunately, you did not pass this time.")
```

Tip: After the colon you must leave an indentation (4 spaces) before the commands that belong to the condition. This indentation is what determines what follows the condition.

Multiple conditions (elif)

```
age = 15
if age >= 18:
    print("You are an adult.")
elif age >= 13:
    print("You are a teenager.")
else:
    print("You are still a child.")
```

Output on the screen:

```
You are a teenager.
```

Logical operators: and / or / not

```
age = 20
has_id = True
if age >= 18 and has_id:
    print("Entry allowed")
if not has_id:
    print("Bring your ID")
```

- `and`: both conditions must be true.
- `or`: it is enough for one of them to be true.
- `not`: reverses the result.

Exercise

- Ask the user for a score and print the grade: Excellent (90 and above), Very Good (80 and above), Good (65 and above), Pass (50 and above), otherwise Fail.

Lesson 5: The for loop

Sometimes we need to repeat a command several times. Instead of writing the code hundreds of times, we use loops.

The range() function

- `range(5)` generates 0, 1, 2, 3, 4 (it starts from 0 and stops before 5).
- `range(1, 6)` generates 1, 2, 3, 4, 5 (a start and an end; the end is not included).
- `range(0, 10, 2)` generates 0, 2, 4, 6, 8 (the third number is the step).

```
for i in range(3):
    print("I love programming")
```

Output on the screen:

```
I love programming
I love programming
I love programming
```

Printing the counter value

```
for number in range(1, 6):
    print(number)
```

Output on the screen:

```
1
2
3
4
5
```

Example: calculating the sum

```
total = 0
for n in range(1, 6):
    total = total + n
print("Sum:", total)
```

Output on the screen:

```
Sum: 15
```

Exercise

- Print the multiplication table (from 1 to 10) for a number entered by the user.

Lesson 6: The while loop, break and continue

The `while` loop repeats the code as long as the condition is true. We use it when we do not know in advance how many times the code must repeat.

```
count = 1
while count <= 5:
    print("Number:", count)
    count += 1
```

Output on the screen:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

Tip: If you forget to change the variable inside the loop (here `count += 1`), the loop becomes infinite. To stop the program, press Ctrl + C.

The break command: exiting the loop

```
while True:
    password = input("Enter the password: ")
    if password == "1234":
        print("Logged in successfully")
        break
    print("Wrong password, try again")
```

The continue command: skipping the current iteration

```
for i in range(1, 8):
    if i % 2 == 0:
        continue
    print(i)
```

Output on the screen:

```
1
3
5
7
```

Tip: If you use `while True` for continuous work, such as monitoring something, add `time.sleep(1)` inside it (after `import time`) so the program does not use up the entire processor.

Exercise

- Write a secret-number guessing game: the secret number is 7, and the program keeps asking the user until they guess it.

Lesson 7: Working with strings

A string in Python is a sequence of characters, and any character can be accessed by its position (counting starts from 0).

```
text = "Python"
print(text[0])    # P
print(text[-1])   # n (the last one)
print(text[0:3])  # Pyt (from 0 up to before 3)
print(len(text))  # 6
```

Common string functions

```
print("python".upper())   # PYTHON
print("PYTHON".lower())   # python
print("  hello  ".strip())  # hello
print("a,b,c".split(","))  # ['a', 'b', 'c']
print("I like cats".replace("cats", "Python"))  # I like Python
print("Py" in "Python")   # True
```

String formatting (f-strings)

The easiest way to combine variables inside text is to put the letter `f` before the quotation mark and write the variable between `{ }`.

```
name = "Sara"
age = 22
print(f"My name is {name} and I am {age} years old")
print(f"In 5 years I will be {age + 5}")
```

Output on the screen:

```
My name is Sara and I am 22 years old
In 5 years I will be 27
```

Exercise

- Ask the user for a name and print it in capital letters, then print the number of its letters.

Lesson 8: Lists

A list stores multiple items in a single variable. It is created with square brackets `[ ]`, with the items separated by commas.

```
fruits = ["apple", "banana", "orange", "strawberry"]
print(fruits)
```

Accessing items (index)

Counting starts from 0, not 1. The first item has index 0, the second 1, and so on. The index -1 means the last item.

```
fruits = ["apple", "banana", "orange"]
print(fruits[0])   # apple
print(fruits[2])   # orange
print(fruits[-1])  # orange
```

Modifying, adding, and deleting

```
names = ["Ahmed", "Khaled"]
names[0] = "Mohammed"   # modify
names.append("Sara")    # add at the end
print(names)            # ['Mohammed', 'Khaled', 'Sara']
names.remove("Khaled")  # delete by value
print(names)            # ['Mohammed', 'Sara']

numbers = [5, 2, 9, 1]
numbers.sort()
print(numbers)          # [1, 2, 5, 9]
last = numbers.pop()    # removes the last item and returns it
print(last)             # 9
print(len(numbers))     # 3
```

Looping over a list with for

```
cars = ["Toyota", "Ford", "Mercedes"]
for car in cars:
    print("I love the car:", car)
```

Output on the screen:

```
I love the car: Toyota
I love the car: Ford
I love the car: Mercedes
```

Exercise

- Ask the user to enter 5 numbers and store them in a list, then print the largest using `max()` and their sum using `sum()`.

Lesson 9: Tuples and sets

The Tuple

It is similar to a list, but it cannot be modified after it is created. It is written with regular parentheses `( )`. It is suitable for fixed data such as coordinates.

```
point = (10, 20)
print(point[0])  # 10

# Unpacking the values into variables
x, y = point
print(x, y)  # 10 20
```

The Set

A collection of items with no duplicates and no fixed order. It is written with curly braces `{ }`.

```
colors = {"red", "green", "red"}
print(len(colors))  # 2 (the duplicate is removed)
colors.add("blue")
print("green" in colors)  # True

numbers = [1, 2, 2, 3, 3, 3]
unique = sorted(set(numbers))
print(unique)  # [1, 2, 3]
```

Tip: Use a set when you want to remove duplicates or quickly check whether an item exists.

Exercise

- You have the list `[1, 7, 9, 4, 7, 4]`. Print only the unique numbers, sorted.

Lesson 10: Dictionaries

A dictionary stores data as key-value pairs, for example linking "name" to "Ahmed" and "age" to 28. It is created with curly braces.

```
employee = {
    "name": "Ahmed",
    "age": 28,
    "job": "programmer"
}
print(employee)
```

Accessing, modifying, and adding

```
print(employee["name"])  # Ahmed
employee["age"] = 29     # modify
employee["city"] = "Setif"  # add a new key
print(employee.get("salary", "not found"))  # not found
```

Tip: Accessing a key that does not exist using `[ ]` causes a `KeyError`, whereas `.get()` returns a default value instead of an error.

Looping over a dictionary

```
for key, value in employee.items():
    print(key, ":", value)
```

Output on the screen:

```
name : Ahmed
age : 29
job : programmer
city : Setif
```

The difference between a list and a dictionary

Comparison	List	Dictionary
Brackets	Square [ ]	Curly { }
Data organization	Sequential items	Key-value pairs
Accessing an item	By position: list[0]	By key: dict["name"]

Exercise

- Create a dictionary for a book (title, author, year of publication), then print all its information using a loop.

Lesson 11: Functions

If you need to repeat code in different places, put it in a named box and call it when needed. This box is the function, and we define it with `def`.

```
def greet():
    print("Welcome!")
    print("We wish you a happy day.")

greet()  # calling the function
```

Tip: The code inside a function runs only when it is called by its name with parentheses.

Parameters

```
def greet_user(username):
    print("Hello,", username)

greet_user("Saleh")
greet_user("Fatima")
```

Output on the screen:

```
Hello, Saleh
Hello, Fatima
```

Returning values (return)

Sometimes we want the function to calculate something and return the result so we can use it elsewhere, instead of printing it directly.

```
def add_numbers(num1, num2):
    result = num1 + num2
    return result

total = add_numbers(5, 10)
print("The sum of the two numbers is:", total)
```

Output on the screen:

```
The sum of the two numbers is: 15
```

Default values

```
def power(base, exp=2):
    return base ** exp

print(power(5))     # 25
print(power(2, 3))  # 8
```

Variable scope

A variable defined inside a function exists only inside it and cannot be used outside it.

```
def show():
    secret = 10
    print(secret)

show()  # 10
# print(secret)  # NameError
```

Exercise

- Write a function `is_even(n)` that returns True if the number is even, otherwise False.

Lesson 12: Modules and libraries (import)

Python comes with ready-made libraries (modules) that we call with the `import` command. There are also external libraries that are installed with the command `pip install library_name` in the command prompt.

```
import math
import random

print(math.sqrt(16))  # 4.0
print(math.pi)        # 3.14159...
print(random.randint(1, 10))  # a random number from 1 to 10
```

Importing part of a library

```
from math import sqrt
print(sqrt(25))  # 5.0
```

Useful standard libraries

- `math`: mathematical functions.
- `random`: random numbers and choices.
- `datetime`: dates and times.
- `time`: waiting with `time.sleep(2)` and measuring time.
- `json`: working with JSON files.
- `os`: working with the operating system and folders.

Exercise

- Write a program that picks a random number between 1 and 100 and prints it.

Lesson 13: Error handling (try / except)

If the user types "twenty" instead of "20", `int()` will fail and the program will crash. To protect the program we use `try` and `except`.

- `try:` we put the code in which an error might occur.
- `except:` the alternative code that runs when an error occurs, instead of the program crashing.

```
try:
    age = int(input("Please enter your age in digits: "))
    print("Your age was accepted:", age)
except ValueError:
    print("Error! Write whole numbers only (for example: 25)")
```

Tip: It is always better to specify the type of error (such as `ValueError`) instead of writing a bare `except:`, because a general `except:` hides other errors you did not intend to catch.

Handling specific errors

```
try:
    num1 = int(input("Enter the numerator: "))
    num2 = int(input("Enter the denominator: "))
    print("The result of the division is:", num1 / num2)
except ZeroDivisionError:
    print("Math error: you cannot divide by zero!")
except ValueError:
    print("Input error: you must enter numbers only!")
```

else and finally

- `else`: runs if no error occurred.
- `finally`: always runs, whether or not an error occurred (useful for closing files, for example).

Repeating the question until the user enters a valid value

```
while True:
    try:
        age = int(input("Enter your age: "))
        break
    except ValueError:
        print("Enter a whole number only")
print("Your age:", age)
```

Exercise

- Ask for two numbers and divide the first by the second, handling both errors: division by zero and non-numeric input.

Lesson 14: Working with files

We use `open()` to read files and write to them. It is best to use it with `with`, because it closes the file automatically.

Mode	Meaning
"r"	Read (the default)
"w"	Write (erases the old content of the file)
"a"	Append to the end of the file

Writing to a file

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("First line\n")
    f.write("Second line\n")
```

Reading from a file

```
with open("notes.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

Output on the screen:

```
First line
Second line
```

Tip: Always write `encoding="utf-8"` when working with Arabic text so that the letters do not appear garbled.

JSON files

```
import json

data = {"name": "Ahmed", "age": 28}
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False)

with open("data.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded["name"])  # Ahmed
```

Exercise

- Write a program that asks the user for a note and appends it to the end of the file `notes.txt`, then displays all the notes.

Lesson 15: Object-oriented programming (1): Classes and objects

When programs get bigger, it becomes hard to organize them with only variables and functions. OOP turns real-life things into code through two concepts:

- **Class:** the blueprint or template, such as the plan for building a car.
- **Object:** the real thing made from the blueprint, such as the car itself.

```
class Dog:
    breed = "Husky"
    color = "white"

my_dog = Dog()
print("Dog breed:", my_dog.breed)
print("Dog color:", my_dog.color)
```

The constructor method __init__

To make each object unique, we use `__init__`, which runs automatically when the object is created. The word `self` means "this current object."

```
class Car:
    def __init__(self, brand, model_year):
        self.brand = brand
        self.model_year = model_year

car1 = Car("Toyota", 2022)
car2 = Car("Mercedes", 2025)
print("First car:", car1.brand)
print("Second car's model year:", car2.model_year)
```

Functions inside a class (methods)

```
class Student:
    def __init__(self, name):
        self.name = name

    def study(self):
        print(self.name, "is studying Python lessons now!")

student1 = Student("Omar")
student1.study()
```

Output on the screen:

```
Omar is studying Python lessons now!
```

Example: a bank account

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}, balance: {self.balance}")

account = BankAccount("Layla", 1000)
account.deposit(500)
```

Output on the screen:

```
Deposited 500, balance: 1500
```

Exercise

- Add a `withdraw(amount)` method to the bank account that withdraws the amount, and prints an error message if the balance is insufficient.

Lesson 16: Object-oriented programming (2): Inheritance

Inheritance allows a new class (the subclass) to inherit the properties and methods of an existing class (the base class), then add to them or modify them, so we do not repeat code.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(self.name, "makes a sound")

class Dog(Animal):
    def speak(self):
        print(self.name, ": Woof woof!")

class Cat(Animal):
    def speak(self):
        print(self.name, ": Meow!")

animals = [Dog("Rex"), Cat("Lulu"), Animal("Animal")]
for a in animals:
    a.speak()
```

Output on the screen:

```
Rex : Woof woof!
Lulu : Meow!
Animal makes a sound
```

This is called polymorphism: the same call, `speak()`, gives different behavior depending on the type of the object.

Calling the parent constructor with super()

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, grade):
        super().__init__(name)
        self.grade = grade

s = Student("Sami", 85)
print(s.name, s.grade)  # Sami 85
```

Exercise

- Create a class `Shape` and two subclasses, `Rectangle` and `Circle`, each with an `area()` method that calculates its area.

What comes after the basics?

After mastering the previous lessons, complete these projects yourself without copying the solutions, because programming is mastered through practice:

- Number guessing game: `random`, `while`, and `if`.
- A calculator with a menu of choices: functions, conditions, and error handling.
- A task manager app: lists, dictionaries, and saving to a JSON file.
- A bank account system: classes and inheritance.

Then choose a path according to your goal:

- Data analysis: pandas and matplotlib.
- Web development: Flask or Django.
- Automation: requests, selenium, and working with files.
- Advanced topics: generators, decorators, testing (pytest), and asynchronous programming (asyncio).